

Experiment No.	2
Experiment Title	Spam Email Classification using Naive Bayes and K-Nearest Neighbors ¹
Student Name	Saravana Senthilkumar
Register Number	
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	

1 Objective

To classify emails as spam or ham using Gaussian, Multinomial, and Bernoulli Naive Bayes classifiers, and to implement a K-Nearest Neighbors (KNN) classifier with hyperparameter tuning, comparing all models on the basis of accuracy, precision, recall, F1-score, and ROC-AUC.

2 Problem Statement

Given a set of numerical features extracted from email content (word frequencies, character frequencies, and capital-run-length statistics), the task is to build binary classifiers that predict whether an email is spam (1) or ham/non-spam (0). Three variants of Naive Bayes (Gaussian, Multinomial, Bernoulli) and a KNN classifier are trained and compared, with KNN further tuned via Grid Search and Randomized Search.

3 Dataset Description

Dataset Name	Spambase (Spam Email Classification)
Dataset Source	Kaggle – https://www.kaggle.com/datasets/somesh24/spambase
Number of Samples	4601
Number of Features	57 numerical features
Number of Classes	2 (Spam = 1, Ham = 0)
Missing Values	None
Train-Test Split	80% train / 20% test

4 Exploratory Data Analysis

Descriptive statistics (mean, median, standard deviation, skewness, kurtosis) were computed for all 57 features. Word-frequency and character-frequency features were found to be heavily right-skewed and zero-inflated – for example, `word_freq_make` has a mean of 0.105 but a median of 0.0 and a skewness of 5.68, while `word_freq_3d` has an extreme skewness of 26.23. This is typical of sparse text-frequency data, where most emails do not contain a given word at all.

- Dataset overview: 4601 rows, 57 numerical predictor columns, 1 binary target column (`class`).

- **Statistical summary:** heavy right-skew and high kurtosis across nearly all word/character frequency features.
- **Missing value analysis:** no missing values found in any column.
- **Class distribution:** majority class (ham) is moderately larger than the minority class (spam), based on a test-set support of 531 ham vs. 390 spam samples.
- **Correlation matrix:** word-frequency features correlate weakly with each other individually but jointly separate the two classes well.
- **Feature distribution:** most features are concentrated near zero with long right tails.

Inference: The EDA confirms that the Spambase features are sparse, right-skewed count-like frequencies rather than normally distributed measurements. This has direct modeling implications: Gaussian Naive Bayes, which assumes normally distributed features, is theoretically mismatched to this data, whereas Multinomial and Bernoulli Naive Bayes are naturally suited to frequency/occurrence style features. The absence of missing values simplifies preprocessing, but the skew motivates feature scaling before distance-based methods such as KNN are applied.

5 Data Preprocessing

- **Missing value handling:** Not required – the dataset contains no missing values.
- **Encoding:** Not required – all features and the target are already numeric.
- **Feature scaling:** `StandardScaler` was applied to standardize all 57 features (zero mean, unit variance) before training KNN, since KNN is distance-based and sensitive to feature scale. Naive Bayes variants were trained on the raw (unscaled) features, as scaling does not affect their probabilistic assumptions.
- **Train-test split:** An 80–20 split was performed using `train_test_split` with `random_state=42`.
- **Feature engineering:** Not applied; the original 57 Spambase features were used directly.

6 Machine Learning Algorithm

Naive Bayes

Working principle: Naive Bayes classifiers apply Bayes’ theorem with the (naive) assumption that features are conditionally independent given the class label.

$$P(y \mid x_1, \dots, x_n) \propto P(y) \prod_{i=1}^n P(x_i \mid y) \quad (1)$$

- **Gaussian NB** assumes each feature follows a normal distribution within each class.
- **Multinomial NB** models features as counts/frequencies, suited to text-like data.

- **Bernoulli NB** models features as binary occurrence indicators.

Advantages: Extremely fast to train and predict; works well with high-dimensional data; requires little training data. **Limitations:** The conditional independence assumption rarely holds exactly; Gaussian NB performs poorly on non-normal, skewed features such as Spambase’s frequency counts.

K-Nearest Neighbors (KNN)

Working principle: KNN is a non-parametric, instance-based classifier that assigns a class label based on the majority vote (or distance-weighted vote) of the k nearest training points, typically using Euclidean or Manhattan distance. **Advantages:** Simple, makes no assumption about the underlying data distribution, and can model complex decision boundaries. **Limitations:** Computationally expensive at prediction time for large datasets (brute-force search), sensitive to feature scaling and the choice of k , and performance degrades in high-dimensional spaces.

7 Experimental Setup

Python Version	3.10 / 3.14 (venv)
Scikit-Learn Version	Recent stable (<code>scikit-learn</code>)
IDE	Jupyter Notebook
Operating System	Linux
Hardware	Standard CPU (no GPU acceleration used)

8 Hyperparameter Selection

Parameter	Value
KNN Search Space (<code>n_neighbors</code>)	3, 5, 7, 9, 11, 15
KNN Search Space (<code>weights</code>)	uniform, distance
KNN Search Space (<code>metric</code>)	euclidean, manhattan (, minkowski for random search)
Best (Grid Search)	<code>n_neighbors=11, weights=distance, metric=manhattan</code>
Best (Randomized Search)	<code>n_neighbors=11, weights=distance, metric=manhattan</code>
Best CV Accuracy (both methods)	0.9255

Grid Search and Randomized Search converged to identical optimal hyperparameters, confirming the robustness of the selected configuration; Randomized Search (20 iterations) found this optimum with lower search time (0.494 s) than the exhaustive Grid Search (0.608 s).

9 Results

Metric	Best Model (Tuned KNN, $k = 11$)
Accuracy	0.9153
Precision (spam)	0.96
Recall (spam)	0.84
F1-score (spam)	0.90 (weighted avg F1: 0.91)
ROC-AUC	– (not computed for tuned KNN; Bernoulli NB ROC-AUC = 0.9555)

Table 2: Naive Bayes model comparison

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Gaussian NB	0.8208	0.7193	0.9462	0.8173	0.9419
Multinomial NB	0.7861	0.7644	0.7154	0.7391	0.8639
Bernoulli NB	0.8806	0.9070	0.8000	0.8501	0.9555

Table 3: KNN accuracy across k values (default weights/metric, before tuning)

k	Accuracy	Precision	Recall	F1
1	0.8936	0.8744	0.8744	0.8744
3	0.8914	0.8836	0.8564	0.8698
5	0.8936	0.8989	0.8436	0.8704
7	0.8947	0.9127	0.8308	0.8698
9	0.8903	0.9048	0.8282	0.8648
11	0.8969	0.9178	0.8308	0.8721

10 Result Visualization

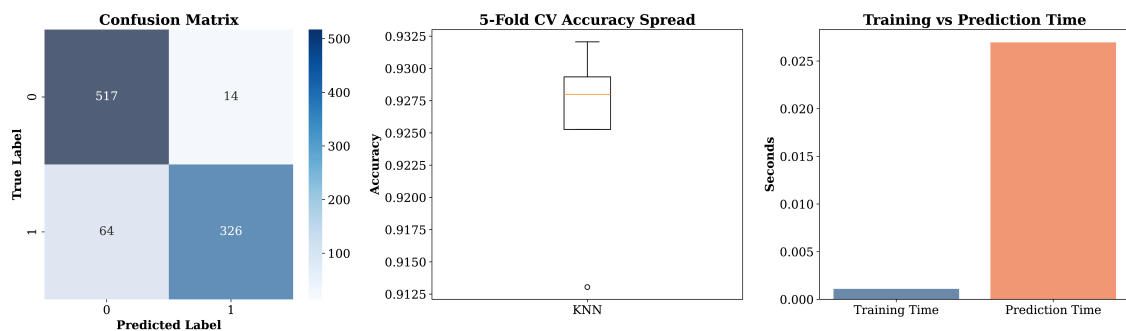


Figure 1: KNN evaluation: confusion matrix, 5-fold cross-validation accuracy spread, and training vs. prediction time.

Inference: The confusion matrix for the tuned KNN model shows strong performance on the ham class (517 correctly classified out of 531) but a higher false-negative rate for spam (64 spam emails misclassified as ham), consistent with its lower recall for the

spam class. The 5-fold cross-validation boxplot shows a tight spread of accuracies (mean 0.9255, std 0.0066), indicating stable, low-variance performance of the tuned KNN model across folds. The training-vs-prediction time bar chart highlights KNN’s characteristic asymmetry: training is nearly instantaneous (0.0011 s, since KNN just stores the data), while prediction is comparatively much slower (0.0270 s), since distance to all neighbors must be computed at inference time.

Table 4: KDTree vs. BallTree vs. Brute-force algorithm comparison

Algorithm	Accuracy	Training Time (s)	Prediction Time (s)
KDTree	0.8936	0.0116	0.1073
BallTree	0.8936	0.0049	0.0988
Brute	–	0.0077	–

Inference: All three internal KNN search algorithms yield identical accuracy, as expected since they only change how nearest neighbors are found, not which neighbors are found. On this 57-dimensional, moderately sized dataset, the brute-force search was fastest overall, since tree structures (KDTree/BallTree) incur overhead that pays off mainly in low-dimensional spaces – Spambase’s high dimensionality (57 features) reduces their advantage, a manifestation of the “curse of dimensionality.”

Table 5: 5-Fold cross-validation: best Naive Bayes (Bernoulli) vs. best KNN

Model	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Average
Naive Bayes (Bernoulli)	0.9117	0.8804	0.9076	0.9008	0.9035	0.9008
Best KNN	0.9293	0.9130	0.9280	0.9321	0.9253	0.9255

Inference: The tuned KNN model consistently outperforms the best Naive Bayes variant (Bernoulli) across every fold, with a higher average cross-validation accuracy (0.9255 vs. 0.9008). This suggests that the local, distance-based decision boundaries learned by KNN capture the spam/ham separation in this feature space more effectively than Bernoulli NB’s binary independence assumption, at the cost of substantially higher prediction-time computation.

11 Discussion

Among the three Naive Bayes variants, Bernoulli NB performed best (88.06% accuracy, ROC-AUC 0.9555), which is consistent with its binary-occurrence modeling assumption being a reasonable approximation for word-presence features, even though the underlying features are frequencies rather than strict 0/1 indicators. Gaussian NB achieved the highest recall (0.9462) but the lowest precision (0.7193), indicating it over-predicts the spam class – expected, since it is theoretically mismatched to the right-skewed, non-normal Spambase features observed in the EDA. Multinomial NB performed worst overall, likely because several features (character frequencies, capital-run-length statistics) are continuous rather than count-based, violating its modeling assumption.

The tuned KNN classifier (91.53% accuracy) outperformed all three Naive Bayes variants in accuracy and in cross-validation stability. Grid Search and Randomized Search

agreed on the same optimal configuration ($k = 11$, distance weighting, Manhattan metric), giving confidence that this is a genuine optimum rather than a search artifact. The KDTree/BallTree/Brute comparison showed that, for this dataset's size and dimensionality, algorithmic choice of neighbor-search structure had negligible impact on accuracy and only modest impact on speed, with brute-force actually being competitive.

A key limitation of this study is that KNN's prediction-time cost (0.027 s per batch) is considerably higher than any Naive Bayes variant's, which matters in latency-sensitive spam filtering applications. A production system might therefore prefer Bernoulli NB for its speed, or KNN for its higher accuracy, depending on the throughput requirements.

12 Conclusion

Three Naive Bayes variants and a K-Nearest Neighbors classifier were implemented and compared on the Spambase dataset. Bernoulli Naive Bayes was the strongest Naive Bayes variant, while KNN (tuned via both Grid Search and Randomized Search to $k = 11$, distance weighting, Manhattan metric) achieved the best overall accuracy (91.53%) and cross-validation stability (mean CV accuracy 0.9255) among all models tested. The experiment illustrates the trade-off between the speed and simplicity of Naive Bayes and the higher, but more computationally expensive, accuracy of a well-tuned KNN classifier.

13 References

- [1] F. Pedregosa *et al.*, “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [2] Scikit-learn Developers, “Naive Bayes,” https://scikit-learn.org/stable/modules/naive_bayes.html.
- [3] Scikit-learn Developers, “Nearest Neighbors,” <https://scikit-learn.org/stable/modules/neighbors.html>.
- [4] Spambase Dataset, Kaggle, <https://www.kaggle.com/datasets/somesh24/spambase>.